

Bioinformática

Módulo 2

Representação computacional de sequências e suas anotações

Algoritmos básicos de processamento: dogma central

BioPython – módulos Seq, SeqIO

Representação de sequências

Computacionalmente, assumiremos que as sequências biológicas são representadas como **strings** (sequências de caracteres)

DNA e RNA: alfabeto com 4 letras: A, C, G, T para DNA e A, C, G, U para RNA

Proteínas representadas por um alfabeto de 20 letras, cada uma representando um aminoácido

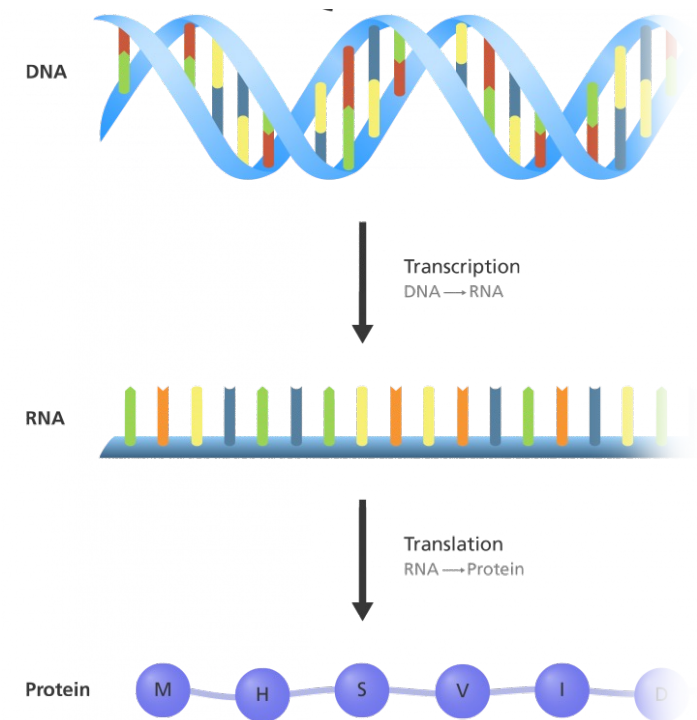
Alfabetos IUPAC

Symbol	Name	Nucleotides represented
A	Adenine	A
C	Cytosine	C
G	Guanine	G
T	Thymine	T
U	Uracil	U

Symbol	Name
A	Alanine
C	Cysteine
D	Aspartic acid
E	Glutamic acid
F	Phenylalanine
G	Glycine
H	Histidine
I	Isoleucine
L	Lysine
M	Methionine
N	Asparagine
P	Proline
Q	Glutamine
R	Arginine
S	Serine
T	Threonine
V	Valine
T	Tryptophan
Y	Tyrosine

Dogma central da Biologia Molecular

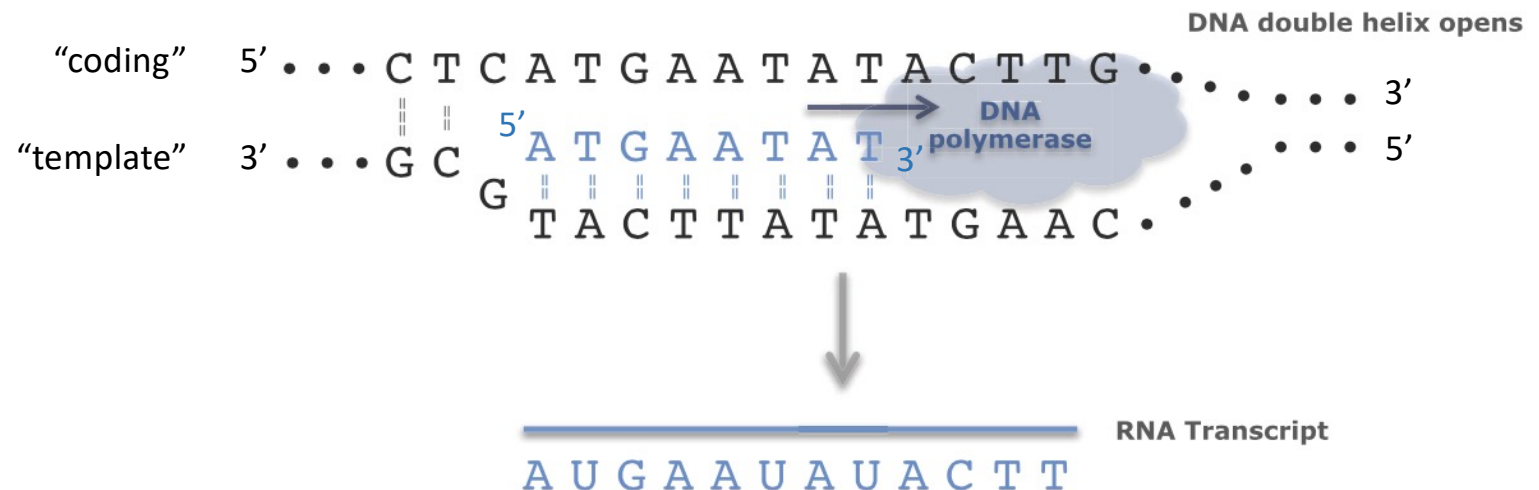
- A síntese de proteínas acontece em dois passos:
 - **Transcrição:** síntese de RNA
 - é o processo de formação do RNA a partir da cadeia de DNA.
 - **Tradução:** síntese de proteína
 - processo onde a informação presente no RNAm (RNA mensageiro) é utilizada pelos ribossomas para sintetizar uma cadeia de aminoácidos.
 - a cada conjunto de 3 acidos nucleicos corresponde 1 aminoácido.



Transcrição

A **transcrição** consta da conversão de uma sequência de DNA, numa sequência de RNA

Dado que as sequências de DNA guardadas nas BDs são de coding DNA (cDNA), lidas na direção 5' → 3', basta alterar todos os T's para U's



Tradução

A **tradução** consta da conversão de uma sequência de cDNA/mRNA, numa sequência de aminoácidos

O processo passa por:

- identificar dos codões (sub-sequências sucessivas de 3 bases)

- converter cada codão no aminoácido respetivo, segundo a tabela de conversão usada pela espécie

- juntar (concatenar) os aminoácidos resultantes pela ordem dada, para criar as possíveis proteínas

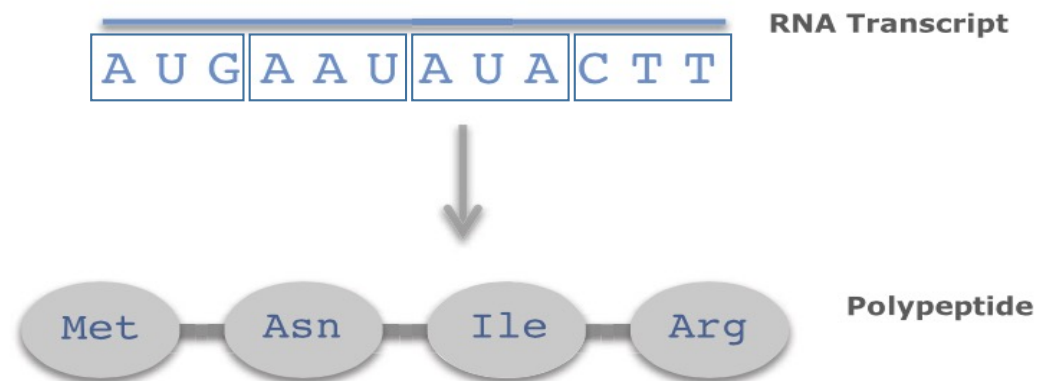


Tabela de conversão: código standard

1st base	2nd base								3rd base
	U		C		A		G		
U	UUU	(Phe/F) Phenylalanine	UCU	(Ser/S) Serine	UAU	(Tyr/Y) Tyrosine	UGU	(Cys/C) Cysteine	U
	UUC		UCC		UAC		UGC		C
	UUA		UCA		UAA	Stop (Ochre)	UGA	Stop (Opal)	A
	UUG		UCG		UAG	Stop (Amber)	UGG	(Trp/W) Tryptophan	G
C	CUU	(Leu/L) Leucine	CCU	(Pro/P) Proline	CAU	(His/H) Histidine	CGU	(Arg/R) Arginine	U
	CUC		CCC		CAC		CGC		C
	CUA		CCA		CAA	(Gln/Q) Glutamine	CGA		A
	CUG		CCG		CAG		CGG		G
A	AUU	(Ile/I) Isoleucine	ACU	(Thr/T) Threonine	AAU	(Asn/N) Asparagine	AGU	(Ser/S) Serine	U
	AUC		ACC		AAC		AGC		C
	AUA		ACA		AAA	(Lys/K) Lysine	AGA	(Arg/R) Arginine	A
	AUG ^[A]	(Met/M) Methionine	ACG		AAG		AGG		G
G	GUU	(Val/V) Valine	GCU	(Ala/A) Alanine	GAU	(Asp/D) Aspartic acid	GGU	(Gly/G) Glycine	U
	GUC		GCC		GAC		GGC		C
	GUA		GCA		GAA	(Glu/E) Glutamic acid	GGA		A
	GUG		GCG		GAG		GGG		G

Complemento inverso

Para se obter a sequência **complementar** de uma sequência de DNA deve-se fazer a seguinte conversão, posição a posição:

A => T; T => A; C=> G; G => C

Para manter a leitura na mesma direção a sequência complementar deverá ser invertida, obtendo-se assim o **complemento inverso**.

Permite verificar qual será a outra cadeia do DNA (a que não está guardada nas BDs)



Reading Frames - RFs

Cada sequência de DNA poderá ter **seis frames de leitura** (Reading Frames), na procura de possíveis proteínas codificadas:

Na sequência original podemos começar a ler nas posições 1, 2 ou 3, tendo assim 3 sequências distintas de aminoácidos
Na sequência complementar inversa podemos começar a ler nas posições 1, 2 ou 3.

Assim, de uma sequência de DNA podem extrair-se 6 possíveis sequências de aminoácidos.

Possíveis proteínas

Todas as possíveis proteínas iniciam-se pelo aminoácido Metionina (M) e terminam com stop (representado por _) – **M** _

O stop não pode ocorrer no interior de uma proteína; a Metionina por sua vez pode ocorrer dentro de uma proteína

Seguindo a regra anterior, é possível extrair de uma sequência de aminoácidos todas as possíveis proteínas no seu interior (a haver alguma)

Podemos também aplicar o mesmo processo a todas as reading frames geradas por uma sequência de DNA, obtendo assim as Open Reading Frames

Obviamente, quanto maior for a potencial proteína, maior será a probabilidade de ser realmente uma proteína

Formatos de sequências

- Por razões históricas, as BDs de sequências permitem a visualização (ou exportação) dos seus registos em *flat files* (texto) com uma dada estrutura
- Vários formatos distintos são usados pelas BDs e pelas ferramentas existentes
- Formatos usados pelo NCBI, EBI e DDBJ são muito semelhantes

Formato FASTA

- Linha de definições/ comentários iniciada com >
- Esta linha pode incluir vários identificadores de BDs e identificação da sequência
- Seguido de várias linhas com a sequência

.fna

```
>gi|1322283|gb|U54469.1|DMU54469 Drosophila melanogaster (...)  
CGGTTGCTTGGGTTTTATAACATCAGTCAGTGACAGGCATTTCCAGAGTT (...)  
GCTGCCTTTGGCCACCAAAATCCCAAACCTTAATTAAAGAATTAAATAATT (...)  
TAACCTACGCAGCTTGAGTGCGTAACCGATATCTAGTATACATTTTCGATA (...)
```

.faa

```
>gi|1322285|gb|AAC03525.1| eukaryotic initiation factor 4E-I  
[Drosophila melanogaster]  
MQSDFHRMKNFANPKSMFKTSAPSTEQGRPEPPTSAAAPAEAKDVKPKEDPQET  
GEPAGNTATTTAPAGDDAVRTEHLYKHPLMNVWTLWYLENDRSKSWEDMQNEI  
TSFDTVEDFWSLYNHIKPPSEIKLGSDYSLFKKNIRPMWEDAAN (...)
```

NCBI- format GenBank

 Resources ▾ How To ▾ Sign in to NCBI

Nucleotide

Nucleotide ▾

Search

Advanced

Help

[Learn more](#) about upcoming changes to the Nucleotide, EST, and GSS databases.

GenBank ▾

Send to: ▾

Homo sapiens homeobox A9 (HOXA9), RefSeqGene on chromosome 7

NCBI Reference Sequence: NG_029923.1

[FASTA](#) [Graphics](#)

Go to: ▾

LOCUS

NG_029923

3093 bp

DNA

linear

PRI 13-SEP-2017

DEFINITION

Homo sapiens homeobox A9 (HOXA9), RefSeqGene on chromosome 7.

ACCESSION

[NG_029923](#) REGION: 5001..8093

VERSION

NG_029923.1

KEYWORDS

RefSeq; RefSeqGene.

SOURCE

Homo sapiens (human)

ORGANISM

[Homo sapiens](#)
Eukaryota; Metazoa; Chordata; Craniata; Vertebrata; Euteleostomi;
Mammalia; Eutheria; Euarchontoglires; Primates; Haplorrhini;
Catarrhini; Hominidae; Homo.

COMMENT

REVIEWED [REFSEQ](#): This record has been curated by NCBI staff. The reference sequence was derived from [AC004080.2](#). This sequence is a reference standard in the [RefSeqGene](#) project.

Summary: In vertebrates, the genes encoding the class of transcription factors called homeobox genes are found in clusters named A, B, C, and D on four separate chromosomes. Expression of these proteins is spatially and temporally regulated during embryonic development. This gene is part of the A cluster on chromosome 7 and encodes a DNA-binding transcription factor which

Change region shown

☐ Whole sequence

☒ Selected region

from: 5001 to: 8093

Update View

Customize view

Analyze this sequence

Run BLAST

Pick Primers

Highlight Sequence Features

Find in this Sequence

Related information

Protein

Taxonomy

Components (Core)

Gene

Formato GenBank

- Começa com um header iniciado pela palavra chave LOCUS e que tem um conjunto de outros campos identificados por um título em maiúsculas
- No meio tem a tabela de “FEATURES” com a anotação, i.e. a informação biológica relevante
- No final tem a sequência iniciada pela palavra ORIGIN; no início de cada linha tem o nº da posição
- Cada registo termina com // (terminador)

```

LOCUS       NT_037436                64721 bp    DNA    linear
DEFINITION  Drosophila melanogaster chromosome 3L.
ACCESSION   NT_037436 REGION: complement(1036369..1101089)
VERSION     NT_037436.4
DBLINK      BioProject: PRJNA164
            BioSample: SAMN02803731
            Assembly: GCF_000001215.4
KEYWORDS    RefSeq.
SOURCE      Drosophila melanogaster (fruit fly)
  ORGANISM  Drosophila melanogaster
            Eukaryota; Metazoa; Ecdysozoa; Arthropoda; Hexapoda;
  
```

```

FEATURES             Location/Qualifiers
     source            1..64721
                       /organism="Drosophila melanogaster"
                       /mol_type="genomic DNA"
                       /db_xref="taxon:7227"
                       /chromosome="3L"
                       /genotype="y[1]; Gr22b[1] Gr22d[1] cn[1] CG33964[R4.2]
                       bw[1] sp[1]; LysC[1] MstProx[1] GstD5[1] Rh6[1]"
     gene              1..64721
                       /gene="bab1"
                       /locus_tag="Dmel_CG9097"
                       /gene_synonym="anon-W00118547.639; bab; BAB; BAB-1; bab-1"
                       Bab1; BAB1; bric-a-brac; CG13910; CG9097; Dmel\CG9097"
                       /note="bric a brac 1"
                       /gen_map="3-0.5 cM"
                       /map="61E2-61F1"
                       /db_xref="FLYBASE:FBgn0004870"
  
```

```

ORIGIN
1   ctcgtgctga gaagactttt ggccctgttt gttggccaat acagccatag aacactga
61   ccgaacaga aacagaaacg gaactgaaag cacacactga ataactctgg acacgacg
121  gagaaccgaa tcgaactgaa agaacgagcg gccgggctaa ttgccaaatg gccacgta
181  gcgcgtaacg taacgaatac gttaaacacc catacacaca cgcccgagaca cacacaca
241  cacacacaca cacagagaga cagaggcagc cacacagata cgcacatgta catgccca
301  ctcagttgag cgtcgtaatt gtcgcggatt cttttgaatg cttattcctt aaacgcta
361  tgaagagcga atcgcgtaac gaataacgat cgcgtaacgt ttacgatcgc cggtgatc
421  acgatcaacg atcaacgata agctgcagac aaaatggcgt cggcgcaggc ggagacga
481  gtcggcttgg cgtccgaaca gggaccagtg gtcagaggc agcgcaaggg gacgggat
541  ggcggcgttt ggcgaagcga taacagaagg tgcggcagtc agcaggagga gaagcgtg
  
```

Formato GenBank

LOCUS:

LOCUS DMU54469 2881 bp DNA linear INV 22-FEB-1998

LOCUS name Tamanho da sequência tipo divisão Data submissão

Divisões do NCBI:

BCT – bactérias

INV – invertebrados

MAM – outros mamíferos

PRI – primatas

PLN – plantas

...

DEFINITION:

DEFINITION Drosophila melanogaster eukaryotic initiation factor 4E (eIF4E) gene, alternative splice products, complete cds.

Na linha de definição temos um sumário do conteúdo biológico do registo

Formato GenBank

ACCESSION:

ACCESSION U54469

Código do registo (chave primária)

VERSION:

U54469.1 GI:1322283

Acession.version

GI:geninfo identifier; cada versão de um registo tem um GI diferente

KEYWORDS:

KEYWORDS .

Este campo serve para colocar palavras chave sobre o registo. O seu uso é desencorajado por muitos.

SOURCE:

Organismo e informação taxonómica

SOURCE *Drosophila melanogaster* (fruit fly)

ORGANISM [Drosophila melanogaster](#)

Eukaryota; Metazoa; Arthropoda; Hexapoda; Insecta;

Pterygota; Neoptera; Endopterygota; Diptera; Brachycera;

Muscomorpha; Ephydroidea; Drosophilidae; *Drosophila*.

Formato GenBank

REFERENCE:

Referências bibliográficas relacionadas com o registo

REFERENCE 1 (bases 1 to 2881)

AUTHORS Lavoie,C.A., Lachance,P.E., Sonenberg,N. and Lasko,P.

TITLE Alternatively spliced transcripts from the Drosophila eIF4E gene
produce two different Cap-binding proteins

JOURNAL J. Biol. Chem. 271 (27), 16393-16398 (1996)

PUBMED [8663200](#)

(...)

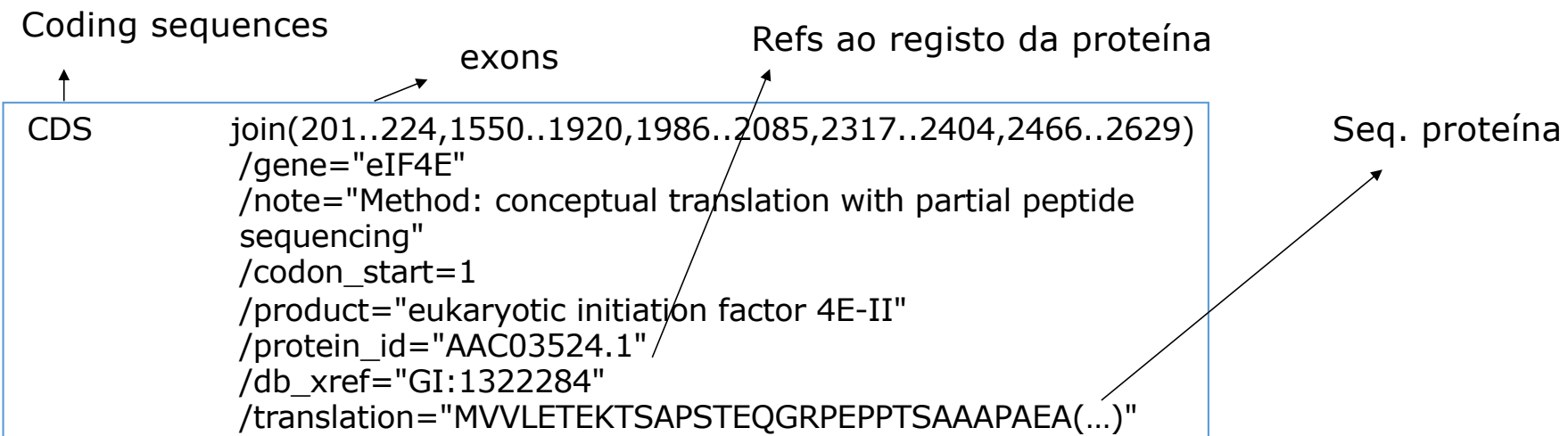
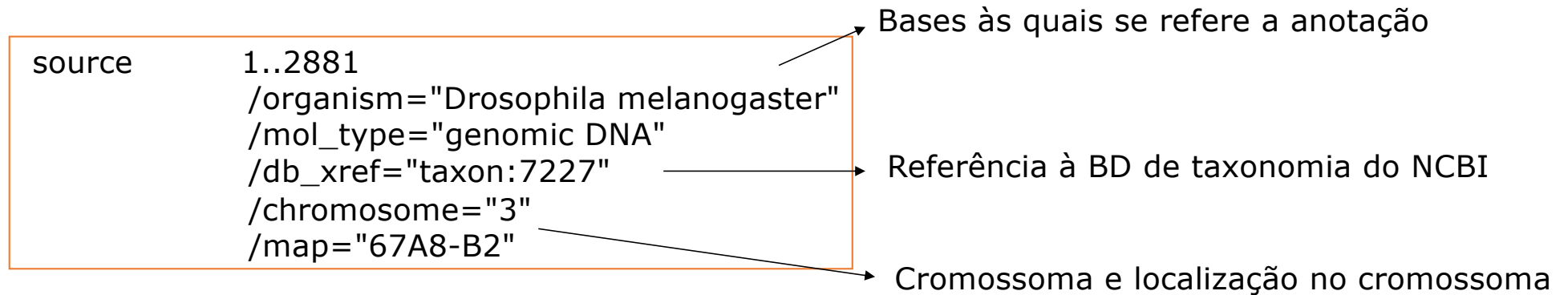
FEATURES:

Esta secção do registo contém as anotações biológicas sendo iniciada com a palavra chave FEATURES. Está organizada em pares chave/ localização:

Na 1ª coluna temos as chaves (tipo de anotação) que pode tomar valores como: source, gene, mRNA, CDS, etc.

Na 2ª colunas temos a informação respeitante a esta chave.

Formato GenBank



BioPython

Baseado no tutorial de BioPython em biopython.org
<http://biopython.org/DIST/docs/tutorial/Tutorial.html>

O que é o BioPython

- Conjunto de recursos escritos na linguagem Python para a abordagem de problemas Bioinformáticos
- Repositório de software livre
- Inclui módulos, scripts e a respetiva documentação
- URL: www.biopython.org
- Repositórios semelhantes existem para outras linguagens: *BioPerl*, *BioJava*, *BioRuby* todos geridos pela Open Bioinformatics Foundation

Principais funcionalidades do BioPython

- **Parsing** de ficheiros em formatos importantes para a Bioinformática:
 - Formatos: FASTA, GenBank, PubMed, SwissProt, Unigene, SCOP, etc.
 - Resultado de ferramentas: Blast, ClustalW, etc.
 - Possibilidade em muitos casos de iterar sobre registos, nos casos em que os ficheiros representam múltiplas entidades
- **Interface com ferramentas e bases de dados** bioinformáticas
 - Através da Web / web services: Blast, Entrez, PubMed, SwissProt, Prosite, etc
 - Através de instalações locais: Blast, ClustalW, etc.

Principais funcionalidades do BioPython

- Implementação de classes para manipulação de **sequências** e sua **anotação** (features), bem como operações de **processamento** sobre sequências (tradução, transcrição, motifs, etc.)
- Implementação de algoritmos de **alinhamento** de sequências e tratamento de matrizes de scoring
- Ferramentas básicas de **mineração de dados**

Instalação do BioPython

- **Anaconda** – para instalar basta lançar uma linha de comandos (no tab Environments- escolher o ambiente e clicar na seta verde); na linha de comandos digitar “conda install biopython” OU no tab Environments procurar o package nos “Not Installed” e instalar
- Se não usar Anaconda poderá usar normalmente o ***pip***

Instruções detalhadas em:

<https://biopython.org/wiki/Download>

Manipulação de sequências: Seq

- Um dos principais objetos no BioPython é o objeto **Seq**. Este permite guardar sequências e trabalhar sobre elas. Os objetos desta classe são **imutáveis**.
- Exemplo: cria e imprime uma sequência (igual para DNA ou proteínas)

```
>>> from Bio.Seq import Seq
>>> my_seq = Seq("AGTACACTGGT")
>>> my_seq
Seq('AGTACACTGGT')
>>> print(my_seq)
AGTACACTGGT
```

—————→ Cria sequência

—————→ Imprime a
sequência

Manipulação de sequências: Seq

- Podemos criar iteradores sobre sequências semelhantes aos das strings

```
>>> my_seq = Seq("GATCG")
>>> for i in my_seq:
...     print(i)
G
A
T
C
G
```

```
>>> for index, letter in enumerate(my_seq):
...     print( index, letter )
0 G
1 A
2 T
3 C
4 G
```

Manipulação de sequências: Seq

```
>>> len(my_seq)
5
>>> my_seq[2]
'T'
>>> my_seq.count("G")
2
>>> my_seq[-2]
'C'
>>> my_seq.lower()
Seq('gatcg', DNAAlphabet())
>>> "GATC" in my_seq
True
>>> my_seq.find("ATC")
1
```

Outras funções semelhantes às strings

Conversão sequências-> strings

```
>>> str(my_seq)
'GATCG'
```

Manipulação de sequências: Seq

```
>>> my_seq = Seq("GATCGATGGGCCTATATAGGATCG")
>>> my_seq[4:12]
Seq('GATGGGCC')
>>> my_seq[0::3]
Seq('GCTGTAGT')
>>> my_seq[1::3]
Seq('AGGCATGC')
```

Slicing de sequências

```
>>> seq1 = Seq("EVRNAK")
>>> seq2 = Seq("GHERW")
>>> seq1+seq2
Seq('EVRNAKGHERW')
```

Concatenação de sequências

Manipulação de sequências: Seq

- Funções sobre sequências de DNA: complemento inverso

```
>>> coding_dna = Seq("ATGGCCATTGTAATGGGCCGCTAG")
>>> coding_dna
Seq('ATGGCCATTGTAATGGGCCGCTAG')
>>> template_dna = coding_dna.reverse_complement()
>>> template_dna
Seq('CTAGCGGCCCATTACAATGGCCAT')
```

Manipulação de sequências: Seq

- Funções sobre sequências de DNA e RNA: transcrição reversa e tradução

```
>>> messenger_rna = coding_dna.transcribe()  
>>> messenger_rna  
Seq('AUGGCCAUUGUAAUGGGCCGCUAG')  
>>> coding_dna.translate()  
Seq('MAIVMGR*')
```

```
>>> rna_seq = Seq('CGUUUAACU')  
>>> rna_seq.translate()  
Seq('RLT')  
>>> rna_seq.back_transcribe()  
Seq('CGTTTAACT')
```

Tabelas de tradução

Outras tabelas: "Bacterial"

```
>>> from Bio.Data import CodonTable
>>> standard_table =
CodonTable.unambiguous_dna_by_name["Standard"]
>>> mito_table =
CodonTable.unambiguous_dna_by_name["Vertebrate Mitochondrial"]
>>> print (standard_table)
(...)
>>> mito_table.stop_codons
['TAA', 'TAG', 'AGA', 'AGG']
>>> mito_table.forward_table["ACG"]
'T'
```

```
>>> coding_dna2 = "ATGGATACCAGA"
>>> coding_dna2.translate()
Seq('MDTR')
>>> coding_dna2.translate(table="Vertebrate
Mitochondrial")
Seq('MDT*')
```

Anotação de sequências: objeto SeqRecord

- Permite associar às sequências biológicas aspectos relacionados com a sua anotação, i.e. características das sequências e das entidades que estas representam
- Objeto **SeqRecord** é o tipo base para o I/O de sequências (a tratar mais adiante)

Campos do SeqRecord

- Um objeto ***SeqRecord*** tem os seguintes campos:
 - **seq**: a sequência propriamente dita
 - **id**: identificador da sequências
 - **name**: nome da sequência
 - **description**: descrição
 - **letter_annotations**: anotações por letra (posição) da sequência
 - **annotations**: anotações globais da sequência (não estruturadas)
 - **features**: anotações estruturadas da sequência (lista de objetos ***SeqFeature***)
 - **dbxrefs**: referências a bases de dados (externas)

Criação de um SeqRecord manualmente

```
>>> from Bio.Seq import Seq
>>> seq = Seq("ATGAATGATAGCTGAT")
>>> from Bio.SeqRecord import SeqRecord
>>> seqRec = SeqRecord(seq)
>>> seqRec.id
'<unknown id>'
>>> seqRec.id = "AB12345"
>>> seqRec.id
'AB12345'
>>> seqRec.description = "Minha sequencia"
>>> seqRec.description
'Minha sequencia'
>>> seqRec.seq
Seq('ATGAATGATAGCTGAT')
>>> seqRec.annotations["role"] = "unknown"
>>> seqRec.annotations
{'role': 'unknown'}
```

Formatos de sequências: objeto SeqIO

- O objeto SeqIO fornece um conjunto de interfaces para trabalhar com vários formatos de sequências biológicas
- As funções disponíveis permitem ler e escrever ficheiros com sequências em diversos formatos distintos
- Normalmente, ao ler as sequências o resultado será um objeto SeqIO, ou um iterador que retorna estes objetos se o ficheiro tiver mais do que uma sequência

SeqIO: função read

- A função read permite ler ficheiros com um único registo
- Exemplo: ficheiro Fasta “NC_005816.fna”:

```
>gi|45478711|ref|NC_005816.1| Yersinia pestis biovar Microtus ... pPCP1, complete sequence
TGTAACGAACGGTGCAATAGTGATCCACACCCAACGCCTGAAATCAGATCCAGGGGGTAATCTGCTCTCC
...
```

http://www.ncbi.nlm.nih.gov/nuccore/NC_005816

```
>>> from Bio import SeqIO
>>> record = SeqIO.read("NC_005816.fna", "fasta")
>>> record
SeqRecord(seq=Seq('TGTAACGAACGGTGCAATAGTGATCCACACCCAACGCCTGA...CTG',
SingleLetterAlphabet()), id='gi|45478711|ref|NC_005816.1|',
name='gi|45478711|ref|NC_005816.1|',
description='gi|45478711|ref|NC_005816.1| Yersinia pestis biovar Microtus ... sequence',
dbxrefs=[])
```

SeqIO: função read (cont.)

```
>>> record.seq
Seq('TGTAACGAACGGTGCAATAGTGATCCACACCCAACGCCTGAAATCAGATCCAGG...CTG',
SingleLetterAlphabet())
>>> record.id
'gi|45478711|ref|NC_005816.1|'
>>> record.name
'gi|45478711|ref|NC_005816.1|'
>>> record.description
'gi|45478711|ref|NC_005816.1| Yersinia pestis biovar Microtus ... pPCP1, complete sequence'
>>> record.dbxrefs
[]
>>> record.annotations
{}
>>> record.letter_annotations
{}
>>> record.features
[]
```

SeqIO: função read

- Exemplo de um ficheiro NCBI Genbank: "NC_005816.gb"

```
LOCUS          NC_005816          9609 bp          DNA    circular BCT 21-JUL-
2008
DEFINITION      Yersinia pestis biovar Microtus str. 91001 plasmid pPCP1, complete
                sequence.
ACCESSION       NC_005816
VERSION         NC_005816.1 GI:45478711
PROJECT         GenomeProject:10638
...
```

```
>>> from Bio import SeqIO
>>> record = SeqIO.read("NC_005816.gb", "genbank")
>>> record
SeqRecord(seq=Seq('TGTAACGAACGGTGCAATAGTGATCCACACCCAACGCCTGA...CTG',
IUPACAmbiguousDNA()), id='NC_005816.1', name='NC_005816',
description='Yersinia pestis biovar Microtus str. 91001 plasmid pPCP1, complete sequence.',
dbxrefs=['Project:10638'])
```

SeqIO: função read (cont)

```
>>> record.seq
Seq('TGTAACGAACGGTGCAATAGTGATCCACACCCAACGCCTGAAATC...CTG',
IUPACAmbiguousDNA())
>>> record.id
'NC_005816.1'
>>> record.name
'NC_005816'
>>> record.description
'Yersinia pestis biovar Microtus str. 91001 plasmid pPCP1, complete sequence.'
>>> len(record.annotations)
11
>>> record.annotations["source"]
'Yersinia pestis biovar Microtus str. 91001'
>>> record.dbxrefs
['Project:10638']
>>> len(record.features)
29
```

Objetos SeqFeature

- Permite guardar informação sobre features (anotações) das sequências de forma estruturada
- Estrutura baseada no formato GenBank / EMBL, na sua tabela de features
- Principais atributos de um objeto SeqFeature:
 - **location**: localização da feature na sequência (pode ser uma posição, um intervalo, etc.)
 - **type**: diz o tipo da feature (string)
 - **qualifiers**: informação adicional (dicionário)

FeatureLocation: exemplo

```
>>> start_pos = SeqFeature.AfterPosition(5)
>>> end_pos = SeqFeature.BetweenPosition(9, left=8, right=9)
>>> my_location = SeqFeature.FeatureLocation(start_pos, end_pos)
```

```
>>> example_parent = Seq("ACCGAGACGGCAAAGGCTAGCATAGGTATGAGACTT")
>>> from Bio.SeqFeature import SeqFeature, FeatureLocation
>>> example_feature = SeqFeature(FeatureLocation(5, 18), type="gene", strand=-1)
>>> feature_seq = example_feature.extract(example_parent)
>>> print (feature_seq)
AGCCTTTGCCGTC
```

SeqFeature: exemplos

```
for feat in record.features:  
    print (feat)
```

Listar todas as features e
devidos atributos

```
featcds = [ ]  
for i in range(len(record.features)):  
    if record.features[i].type == "CDS":  
        featcds.append(i)  
for k in featcds:  
    print (record.features[k].location)  
for k in featcds:  
    print (record.features[k].extract(record.seq))
```

Identificar features
que são do tipo CDS
Identificar a sua
localização
Identificar sub-
sequência do DNA
afetada pela feature

Objeto SeqIO: parse – vários registos

- função: *Bio.SeqIO.parse()*, com argumentos:
 - Handle ou nome do ficheiro
 - Formato do ficheiro (uma string)
 - Opcionalmente, o alfabeto para a sequência a ler
- Retorna iterador sobre objetos *SeqRecord*

```
from Bio import SeqIO
for seq_record in SeqIO.parse("ls_orchid.fasta", "fasta"):
    print (seq_record.id )
    print (seq_record.seq)
    print (len(seq_record))
```

```
from Bio import SeqIO
for seq_record in SeqIO.parse("ls_orchid.gbk", "genbank"):
    print( seq_record.id )
    print (seq_record.seq)
    print (len(seq_record))
```

http://biopython.org/DIST/docs/tutorial/examples/ls_orchid.fasta

SeqIO: exemplos parse

```
>>> from Bio import SeqIO
>>> record_iterator = SeqIO.parse("ls_orchid.gbk", "genbank")
>>> first_record = next(record_iterator)
>>> print (first_record)
ID: Z78533.1
Name: Z78533
Description: C.irapeanum 5.8S rRNA gene and ITS1 and ITS2 DNA.
Number of features: 5
/sequence_version=1
/source=Cypripedium irapeanum
/taxonomy=['Eukaryota', 'Viridiplantae', 'Streptophyta', ..., 'Cypripedium']
/keywords=['5.8S ribosomal RNA', '5.8S rRNA gene', ..., 'ITS1', 'ITS2']
/references=[...]
/accessions=['Z78533']
/data_file_division=PLN
...
```

SeqIO: exemplos parse (cont)

```
>>> print (first_record.annotations["source"])
Cypripedium irapeanum

>>> from Bio import SeqIO
>>> all_species = []
>>> for seq_record in SeqIO.parse("ls_orchid.gbk", "genbank"):
    all_species.append(seq_record.annotations["organism"])
>>> print (all_species)
['Cypripedium irapeanum', 'Cypripedium californicum', ..., 'Paphiopedilum barbatum']
```

Parse de sequências retiradas da web

- Fasta

```
from Bio import Entrez
from Bio import SeqIO
Entrez.email = "...@example.com"
handle = Entrez.efetch(db="nucleotide", rettype="fasta", retmode="text", id="6273291")
seq_record = SeqIO.read(handle, "fasta")
handle.close()
print (seq_record.id, " com ", len(seq_record.features), " features ")))
```

Parse de sequências retiradas da web

- Genbank

```
from Bio import Entrez
from Bio import SeqIO
Entrez.email = "...@example.com"
handle = Entrez.efetch(db="nucleotide", rettype="gb", retmode="text",
id="6273291,6273290,6273289")
for seq_record in SeqIO.parse(handle, "gb"):
    print (seq_record.id, seq_record.description[:100], "...")
    print ("Sequence length: ", len(seq_record))
    print (len(seq_record.features), " features" )
    print ("from: ", seq_record.annotations["source"] )
handle.close()
```

Escrita de sequências: write

- Função *write*: permite escrever sequências para ficheiros em diversos formatos
- Argumentos:
 - Lista de objetos SeqRecord (que representam as sequências e sua anotação que se pretende escrever)
 - Handle ou nome do ficheiro
 - Formato (string)

Escrita sequências: exemplos

```
>>> s1 = SeqRecord(Seq("MMYQQGCFAGGTV"),
                    id = "gi|14150838|gb|AAK54648.1|AF376133_1",
                    description="chalcone synthase [Cucumis sativus]")
>>> s2 = SeqRecord(Seq("MVTVEEFRRQA"),
                    id="gi|13919613|gb|AAK33142.1|",
                    description="chalcone synthase [Fragaria vesca]")
>>> lr = [s1, s2]
>>> from Bio import SeqIO
>>> SeqIO.write(lr, "ex.faa", "fasta")
2
```

Conversão de formatos

```
from Bio import SeqIO
records = SeqIO.parse("ls_orchid.gbk", "genbank")
count = SeqIO.write(records, "my_example.fasta", "fasta")
print ("Convertidos %i registros" % count )
```

```
from Bio import SeqIO
count = SeqIO.convert("ls_orchid.gbk", "genbank", "my_example.fasta", "fasta")
print ("Convertidos %i registros" % count )
```